DEEP LEARNING
UNIT – IV | LONG ANSWERS

CNNs, AlexNet, RNN, BPTT, LSTM, GRU & Restricted Boltzmann Machines

B.Tech V Semester – AI & ML | MIC23 | Subject Code: 23AM5T01

Coverage	Details
Part A – 2 marks	Intentionally excluded as requested
Part B – Long answers	All 9 questions, expanded to 10-mark level
Part C – Numericals	Not included; requested only long answers

Answers follow the exact Unit IV Part B question bank, with definitions, equations, step-by-step working, diagrams/tables and conclusions.

PART B – ESSAY & LONG ANSWER QUESTIONS (10 MARKS EACH)

Q1. Explain Backpropagation Through Time (BPTT) for RNNs with an unrolled graph and mathematical working.

Introduction: Backpropagation Through Time (BPTT) is the training algorithm used for Recurrent Neural Networks. An RNN reuses the same weights at every time step. BPTT unfolds the network across time and applies the ordinary backpropagation algorithm to the resulting computational graph.

1. RNN forward computation

```
a■ = W■■x■ + W■■h■■■ + b■

h■ = ϕ(a■) ; o■ = W■■h■ + b■
```

Here x■ is input, h■ is hidden state, o■ is output, ϕ is an activation function, and W■■, W■■ and W■■ are shared weights.

2. Unrolled computational graph

x■ → [RNN cell] → h■	x■ → [RNN cell] → h■	x■ → [RNN cell] → h■
shared recurrent weights W■■ across all time steps	shared recurrent weights W■■ across all time steps	shared recurrent weights W■■ across all time steps
Output y■ / loss L■	Output y■ / loss L■	Output y■ / loss L■

The repeated cells are not separate networks; they are the same RNN cell applied at different times.

3. Loss calculation

```
L = Σ■■■■ L■
```

For a sequence of length T, the total loss is commonly the sum or average of losses at all time steps.

4. Backward propagation through time

The total loss depends on hidden states from several time steps. Therefore, the gradient for a recurrent weight is obtained by adding its contribution at every time step.

```
∂L/∂W■■ = Σ■■■■ ∂L/∂W■■ | at time t
```

The hidden-state error at time t receives contributions from the current output loss and from later hidden states:

```
δ■ = (W■■■ ∂L■/∂o■ + W■■■ δ■■■) ■ ϕ'(a■)
```

This recurrence is evaluated backwards, starting at the final time step.

5. Parameter updates

```
W ← W − η ∂L/∂W
```

Gradients for shared parameters are accumulated over all unfolded time steps before applying the update.

6. Algorithm steps

• Feed the sequence inputs x■, x■, …, xT into the RNN.

• Compute hidden states and outputs in the forward direction.

• Calculate the loss at each output and combine them.

• Unroll the network over the sequence and propagate errors backwards from T to 1.

• Accumulate gradients for shared weights and biases.

• Update parameters using gradient descent or another optimizer.

7. Advantages and limitations

• Trains RNNs to learn temporal dependencies and sequence patterns.

• Uses the same parameters at every time step, making sequence processing possible.

• Long sequences can cause vanishing or exploding gradients.

• Full BPTT stores many intermediate states and can consume substantial memory; truncated BPTT limits the backward window.

Conclusion: BPTT trains recurrent networks by unfolding them over time, calculating sequence loss and propagating gradients backwards through all time steps.

Q2. Explain LSTM architecture and derive equations for all gates, cell state and hidden state.

Introduction: Long Short-Term Memory (LSTM) is a special RNN architecture designed to learn long-term dependencies. It uses a memory cell and gates that control what information is forgotten, stored and exposed as output.

1. LSTM cell structure

Previous cell state c■■■
Forget gate: decides what to remove
Input gate + candidate: decide what to store
Cell state update → c■
Output gate: selects exposed information
Hidden state h■

At each time step, the LSTM receives current input x■, previous hidden state h■■■ and previous cell state c■■■.

2. Forget gate

```
f■ = σ(W_f [h■■■, x■] + b_f)
```

The sigmoid gate produces values between 0 and 1. Values near 0 forget information; values near 1 retain it.

3. Input gate

```
i■ = σ(W_i [h■■■, x■] + b_i)
```

The input gate determines how much new information should be written into the cell state.

4. Candidate memory

```
■■ = tanh(W_c [h■■■, x■] + b_c)
```

The candidate memory creates new possible content, with values between –1 and +1.

5. Cell state update

```
c■ = f■ ■ c■■■ + i■ ■ ■■
```

The forget gate scales old memory, while the input gate controls the candidate information added to memory. ■ denotes element-wise multiplication.

6. Output gate and hidden state

```
o■ = σ(W_o [h■■■, x■] + b_o)
h■ = o■ ■ tanh(c■)
```

The output gate decides which parts of the updated cell state become the hidden output.

7. Working steps

• Take x■, h■■■ and c■■■ as inputs.

• Calculate forget gate f■ and decide which old memory to retain.

• Calculate input gate i■ and candidate memory ■■.

• Update cell state c■.

• Calculate output gate o■.

• Generate new hidden state h■ and pass h■, c■ to the next time step.

8. Advantages

• Cell state supports information flow over long sequences.

• Gates control memory and reduce the vanishing-gradient problem compared with standard RNNs.

• Useful in language modelling, speech recognition and time-series tasks.

Conclusion: LSTM uses forget, input and output gates to control its memory cell, allowing it to retain useful information over long time intervals.

Q3. Explain GRU architecture, Reset and Update Gates, and compare GRU with LSTM.

Introduction: A Gated Recurrent Unit (GRU) is a gated RNN that captures dependencies in sequential data. It uses two gates—reset and update—and combines the cell and hidden-state roles into a single state.

1. GRU architecture

Current input x■ + previous state h■■■
Reset gate r■ and Update gate z■
Candidate hidden state h■■
Combine old state and candidate
New hidden state h■

2. Reset gate

```
r■ = σ(W_r x■ + U_r h■■■ + b_r)
```

The reset gate controls how much previous information is used when creating the candidate state. A small value reduces the influence of the past.

3. Update gate

```
z■ = σ(W_z x■ + U_z h■■■ + b_z)
```

The update gate controls how much of the previous state is retained and how much new candidate information is incorporated.

4. Candidate hidden state

```
h■■ = tanh(W_h x■ + U_h (r■ ■ h■■■) + b_h)
```

The reset gate modifies the previous hidden state before candidate generation.

5. Final hidden state

```
h■ = (1−z■) ■ h■■■ + z■ ■ h■■
```

This convention defines z■ as the amount of candidate information used. Some textbooks use the complementary convention, but the interpretation remains equivalent.

6. GRU working steps

• Receive current input and previous hidden state.

• Calculate reset and update gates.

• Use reset gate to form the candidate state.

• Use update gate to combine candidate and previous state.

• Pass the new hidden state to the next time step.

7. GRU versus LSTM

Feature	GRU	LSTM
Gates	Reset and Update	Forget, Input and Output
States	One hidden state	Cell state and hidden state
Complexity	Simpler; often fewer parameters	More complex; often more parameters
Memory control	Update gate controls retained/new state	Separate gates control forgetting, writing and output

Training	Can be faster on some tasks	May help when more explicit memory control is useful

8. Applications and limitations

GRUs are used in language processing, speech and time-series prediction. Their simpler structure can reduce computation. Performance depends on the dataset and task; neither GRU nor LSTM is universally better.

Conclusion: GRU uses reset and update gates to control sequential information with a simpler structure than LSTM.

Q4. Explain Standard 2D, Dilated/Atrous, Transposed and Depthwise Separable Convolutions with diagrams and examples.

Introduction: Convolution operations extract spatial features from images and feature maps. Different forms of convolution are used to control receptive field, output resolution and computation.

1. Standard 2D convolution

A small kernel slides over the input and computes weighted sums. The output value is the sum of element-wise products between the kernel and the corresponding input patch.

```
Y(i,j) = Σ■Σ■ X(i+m,j+n) K(m,n)
```

With stride 1 and no padding, a 3×3 input and 2×2 kernel produce a 2×2 output.

Input feature map	Kernel slides over local patches	Output feature map
[1 2 3; 4 5 6; 7 8 9]	[1 0; 0 1]	[6 8; 12 14]

2. Dilated / Atrous convolution

Dilated convolution inserts spaces between kernel elements. It increases the receptive field without proportionally increasing the number of kernel parameters.

```
k_eff = k + (k−1)(d−1)
```

Here k is kernel size and d is dilation rate. For a 3×3 kernel with dilation 2, the effective receptive field is 5×5.

Ordinary 3×3 kernel	Dilated 3×3 kernel (d=2)
Adjacent kernel positions	Kernel positions spaced apart
Smaller receptive field	Larger receptive field without extra kernel weights

It is useful in semantic segmentation and dense prediction, where wider context is required without heavy downsampling.

3. Transposed convolution

Transposed convolution is a learnable operation commonly used for upsampling feature maps. It maps a smaller spatial feature map to a larger one. It is sometimes called deconvolution, although it is not generally the exact mathematical inverse of convolution.

Small feature map	Transposed convolution	Larger feature map
2×2	Learnable upsampling	Larger spatial dimensions

It is used in decoder networks, image generation and segmentation. Output size depends on stride, padding, kernel size and output padding.

4. Depthwise separable convolution

It factorizes a standard convolution into two operations:

• Depthwise convolution: applies one spatial filter independently to each input channel.

• Pointwise convolution: uses 1×1 filters to combine information across channels.

Input channels	Depthwise spatial filtering	Pointwise 1×1 channel mixing	Output channels

This reduces computation and parameters compared with a standard convolution, particularly for many input/output channels. It is used in efficient architectures such as MobileNet.

5. Comparison

Type	Main purpose	Key property

Standard 2D	Spatial feature extraction	Kernel moves over local patches
Dilated	Expand receptive field	Spaces between kernel elements
Transposed	Upsampling	Increases spatial resolution
Depthwise separable	Reduce computation	Depthwise + pointwise steps

Conclusion: Standard convolution extracts local features, dilated convolution captures wider context, transposed convolution upsamples, and depthwise separable convolution reduces computational cost.

Q5. Explain AlexNet architecture and its innovations: ReLU, overlapping pooling, dropout and multi-GPU.

Introduction: AlexNet is a deep convolutional neural network that achieved a major breakthrough in large-scale image classification. It demonstrated the effectiveness of deep CNNs trained with GPUs and large labelled datasets.

1. AlexNet architecture

Input image: 227×227×3 (commonly described)
Conv1: 96 filters, 11×11, stride 4 → ReLU → Max Pool
Conv2: 256 filters, 5×5 → ReLU → Max Pool
Conv3: 384 filters, 3×3 → ReLU
Conv4: 384 filters, 3×3 → ReLU
Conv5: 256 filters, 3×3 → ReLU → Max Pool
Fully connected layers: 4096 → 4096 → class scores
Softmax output

The original network contains five convolutional layers followed by three fully connected layers. Local response normalization was also used in the original design.

2. ReLU activation

```
ReLU(x) = max(0,x)
```

ReLU replaces negative values with zero and keeps positive values unchanged. It is computationally simple and helps reduce saturation-related gradient problems compared with sigmoid or tanh in many settings.

3. Overlapping max pooling

AlexNet used pooling regions whose stride was smaller than the pooling window, so adjacent pooling regions overlapped. This reduced spatial size while retaining strong features and helped reduce overfitting compared with non-overlapping pooling in the original experiments.

4. Dropout regularization

Dropout randomly disables selected neuron activations in fully connected layers during training. It reduces co-adaptation and overfitting by training many thinned subnetworks. During inference, dropout is disabled with the appropriate scaling convention.

5. Multi-GPU parallelization

The original AlexNet was trained using two GPUs. The network computation was divided across GPUs, with communication between parts where needed. This enabled training of a large model within the available hardware constraints.

6. Other important contributions

• Used a deep stack of convolutional layers to learn hierarchical visual features.

• Used data augmentation such as image translations/reflections and colour variation to improve generalization.

• Used GPU acceleration to make large-scale CNN training practical.

7. Advantages and limitations

AlexNet showed that deep CNNs can learn powerful image features and achieve strong classification performance. However, it has many parameters, especially in fully connected layers, and is computationally heavier than later efficient architectures.

Conclusion: AlexNet popularized deep CNNs through ReLU, overlapping pooling, dropout and multi-GPU training, establishing an influential approach to image recognition.

Q6. Compare traditional RNNs, Bidirectional RNNs and Bidirectional LSTMs in architecture, information flow and NLP applications.

Introduction: Recurrent networks process sequences by maintaining hidden states. Traditional RNNs process information in one direction; bidirectional models process it in both directions to use past and future context.

1. Traditional unidirectional RNN

x■ → h■ → h■ → h■ → … → h■
Information flows from earlier to later time steps

At each time step, the hidden state depends on the current input and previous hidden state. It is suitable for tasks where only past context is available, such as next-word prediction.

2. Bidirectional RNN

Forward: x■ → h■■ → h■■ → h■■
Backward: x■ ← h■■ ← h■■ ← h■■
Combine forward and backward states for each position

A bidirectional RNN contains two recurrent layers: one processes the sequence forward and the other backward. Their hidden representations are combined, often by concatenation.

3. Bidirectional LSTM

A Bidirectional LSTM uses LSTM cells in both directions. Each direction has its own memory cell and gates. The two outputs are combined to provide contextual representations that include information from both sides of each token.

Forward LSTM →	Token representation	← Backward LSTM
Past context	Combined representation	Future context

4. Comparison

Aspect	Traditional RNN	Bi-RNN	Bi-LSTM
Directions	One	Two	Two
Context	Past only	Past + future	Past + future with gated memory
Long dependencies	Difficult	Still limited by basic RNN cells	Better handled by LSTM memory
Complexity	Lower	Higher than one-way RNN	Usually highest of these three
Example use	Streaming prediction	Sequence labelling	NER, tagging, speech/text understanding

5. NLP applications

• Part-of-speech tagging and named-entity recognition.

• Text classification and sentiment analysis.

• Speech recognition and sequence labelling.

• Contextual token representation for tasks where the complete input sequence is available.

6. Important limitation

Bidirectional networks require access to the full sequence, so they are not suitable for strictly causal real-time generation where future tokens are unavailable. Traditional unidirectional RNNs can operate incrementally.

Conclusion: Bidirectional RNNs use context from both directions, while Bidirectional LSTMs add gated memory to better capture longer dependencies.

Q7. Explain vanishing/exploding gradients in standard RNNs and why LSTM overcomes them.

Introduction: RNNs reuse recurrent weights at every time step. During BPTT, gradients are repeatedly multiplied by recurrent weight matrices and activation derivatives. Over long sequences, this can make gradients vanish or explode.

1. Gradient propagation

```
∂h■/∂h■■■ = W■■■ × diag(ϕ'(a■))
```

Across many time steps, the gradient contains a product of these terms. The magnitude of this repeated product determines whether the gradient becomes very small or very large.

2. Vanishing gradient

If the repeated factors have magnitudes mostly below 1, the gradient decreases exponentially as it moves backwards through time. Earlier time steps receive almost no learning signal, so the RNN struggles to learn long-term dependencies.

3. Exploding gradient

If the repeated factors have magnitudes mostly above 1, the gradient can grow exponentially. This produces very large parameter updates, unstable training and sometimes numerical overflow.

Vanishing gradient	Exploding gradient
Gradient magnitude → nearly zero	Gradient magnitude → extremely large
Earlier information is not learned	Updates become unstable
Possible remedy: gated memory	Possible remedy: gradient clipping

4. Why standard RNNs suffer

• The same recurrent transformation is applied repeatedly across time.

• Long sequences create long chains of gradient multiplication.

• Sigmoid and tanh derivatives can be small in saturated regions.

• The network may forget information from distant time steps.

5. How LSTM addresses vanishing gradients

LSTM introduces a cell state with an additive update path. The forget and input gates control how memory is retained and written. This creates a path through which information and gradients can travel across many time steps more effectively.

```
c■ = f■ ■ c■■■ + i■ ■ ■■
```

The derivative of the cell state with respect to the previous cell state includes the forget gate term, enabling controlled memory flow instead of repeatedly applying only a dense recurrent transformation.

6. Role of gates

• Forget gate: retains useful old information and removes irrelevant content.

• Input gate: controls new information written into memory.

• Output gate: controls what information is exposed as hidden state.

7. Additional solutions

Gradient clipping limits excessively large gradients. Proper initialization, suitable learning rates and truncated BPTT can also help. LSTM reduces the vanishing-gradient difficulty but does not eliminate every training problem.

Conclusion: Standard RNNs have difficulty learning long dependencies because gradients vanish or explode through repeated time steps. LSTM's gated additive cell-state path helps preserve useful information and gradients.

Q8. Explain RBM architecture, energy function, joint probability and Contrastive Divergence training.

Introduction: A Restricted Boltzmann Machine (RBM) is a generative stochastic neural network that learns a probability distribution over input data. It contains a visible layer and a hidden layer with no connections within the same layer.

1. Bipartite architecture

Visible units: v■ v■ v■ v■
↕ ↕ ↕ ↕
Hidden units: h■ h■ h■
No visible-visible or hidden-hidden connections

Every visible unit may connect to every hidden unit. The absence of within-layer connections makes the graph bipartite and allows conditional units in each layer to be sampled independently when the opposite layer is known.

2. Energy function

```
E(v,h) = −a■v − b■h − v■wh
```

v is the visible vector, h is the hidden vector, W is the weight matrix, and a and b are visible and hidden biases. Lower energy corresponds to higher probability.

3. Joint probability distribution

```
P(v,h) = exp(−E(v,h))/Z

Z = Σ_v Σ_h exp(−E(v,h))
```

Z is the partition function that normalizes probabilities. It sums over all possible visible and hidden configurations.

4. Conditional probabilities

For binary RBMs, the conditional probabilities are:

```
P(h■=1|v) = σ(b■ + Σ■ v■w■■)

P(v■=1|h) = σ(a■ + Σ■ w■■h■)
```

Because there are no same-layer connections, hidden units are conditionally independent given visible units, and vice versa.

5. Contrastive Divergence (CD) training

• Positive phase: take a training example v■ and calculate hidden activation probabilities P(h|v■).

• Sample hidden state h■ from those probabilities.

• Negative phase: reconstruct visible data v■ by sampling P(v|h■), then sample hidden state h■ from P(h|v■).

• Estimate the difference between data-driven and reconstructed associations.

• Update weights and biases using the approximate gradient.

```
ΔW ∝ ■vh■■_data − ■vh■■_reconstruction

Δa ∝ ■v■_data − ■v■_reconstruction

Δb ∝ ■h■_data − ■h■_reconstruction
```

6. Advantages and limitations

• Learns latent features without requiring labelled data.

• Can model probability distributions and generate/reconstruct samples.

• Can be stacked to build deeper generative models.

• Exact likelihood computation is difficult because the partition function is expensive to calculate.

Conclusion: RBM learns data distributions through a bipartite visible-hidden structure. Contrastive Divergence approximates the likelihood gradient using short Gibbs-sampling chains.

Q9. Explain MCMC and Gibbs Sampling in generative energy-based models. Why is exact calculation intractable and sampling necessary?

Introduction: Energy-based generative models assign an energy value to each configuration. Configurations with lower energy receive higher probability. Sampling is needed to estimate model expectations and generate representative configurations.

1. Probability from energy

```
P(x) = exp(−E(x))/Z
```

E(x) is the energy of configuration x and Z is the partition function, which sums exp(−E(x)) over every possible configuration.

2. Why exact calculation is intractable

• The number of possible configurations grows exponentially with the number of variables.

• The partition function requires summing over all possible states.

• For large models, exact expectations and likelihood gradients are computationally expensive.

3. Markov Chain Monte Carlo (MCMC)

MCMC is a family of sampling methods that constructs a Markov chain whose stationary distribution is the target probability distribution. Each new sample depends on the previous state, and repeated transitions allow the chain to explore likely configurations.

Initial state x■
Transition / sampling step
New state x■
Repeat transitions: x■, x■, …
Samples approximate target distribution

4. Gibbs Sampling

Gibbs Sampling is an MCMC method that updates one variable at a time by sampling it from its conditional distribution while keeping the other variables fixed.

• Initialize all variables with a starting configuration.

• Choose one variable and sample it conditioned on the remaining variables.

• Move to the next variable and repeat the conditional sampling.

• Continue multiple sweeps so the chain approaches its stationary distribution.

• Use generated samples to estimate expectations or reconstruct data.

5. Gibbs sampling in an RBM

In an RBM, visible units are conditionally independent given hidden units, and hidden units are conditionally independent given visible units. Therefore, Gibbs sampling alternates between:

```
h■■■ ~ P(h|v■■■) ; v■■■¹■ ~ P(v|h■■■)
```

The alternating steps generate a chain of visible-hidden configurations.

6. Role in model training

Samples from the model distribution are used to estimate negative-phase expectations. These are compared with data-driven expectations to update model parameters. Contrastive Divergence uses a short chain to approximate this process.

7. Limitations

• Samples may be correlated, reducing effective sample count.

• Chains may mix slowly and require many transitions.

• Initial samples may need a burn-in period.

• Approximate sampling can introduce bias into gradient estimates.

Conclusion: MCMC and Gibbs Sampling make learning and generation practical when exact summation over all states is impossible. They approximate the required probability expectations through repeated conditional sampling.

UNIT IV – QUICK REVISION CHECKLIST

• BPTT: unroll sequence, sum time-step losses, backpropagate from T to 1, accumulate shared-weight gradients.

• LSTM: forget gate, input gate, candidate memory, cell update, output gate, hidden state.

• GRU: reset gate, update gate, candidate state, final hidden state.

• Convolutions: standard 2D, dilated, transposed, depthwise separable.

• AlexNet: 5 convolutional + 3 fully connected layers; ReLU, overlapping pooling, dropout, multi-GPU.

• Sequence models: unidirectional RNN, Bi-RNN, Bi-LSTM.

• RNN gradient problem: vanishing/exploding; LSTM uses gated cell state.

• RBM: bipartite visible/hidden layers, energy function, joint probability, CD training.

• MCMC/Gibbs: sampling to estimate distributions when exact partition function is intractable.

Exam writing tip: For a 10-mark answer, include a short introduction, labelled diagram, equations with symbol meanings, working steps, comparison/advantages where relevant, and a conclusion.